

Program 1

```
#include <stdio.h>
int main()
{
printf("welcome to operating system\n");
return 0;
}
```

Output

```
arnav@LAPTOP-BN8992KC:~$ cd OS
arnav@LAPTOP-BN8992KC:~/OS$ nano hello.c
arnav@LAPTOP-BN8992KC:~/OS$ gcc hello.c -o hello
arnav@LAPTOP-BN8992KC:~/OS$ ./hello
welcome to operating system
```

Program 2

```
#include <stdio.h>
int main(){
char name[30];
printf("Enter your name\n");
scanf("%s",name);
printf("Welcome %s\n",name);
return 0;
}
```

Output

```
arnav@LAPTOP-BN8992KC:~/OS$ nano W12.c
arnav@LAPTOP-BN8992KC:~/OS$ gcc W12.c -o W12
arnav@LAPTOP-BN8992KC:~/OS$ ./W12
Enter your name
```

Program 3

```
#include <stdio.h>
#include<unistd.h>
int main(){
printf("Process ID =%d\n",getpid());
return 0;}
```

Output

```
arnav@LAPTOP-BN8992KC:~/OS$ nano W12.c
arnav@LAPTOP-BN8992KC:~/OS$ nano W13.c
arnav@LAPTOP-BN8992KC:~/OS$ gcc W13.c -o W13
arnav@LAPTOP-BN8992KC:~/OS$ ./W13
Process ID =754
arnav@LAPTOP-BN8992KC:~/OS$ |
```

Program 4

```
#include <stdio.h>
#include <unistd.h>
int main(){
fork();
printf("Hello\n");
return 0;
}
```

Output

```
arnav@LAPTOP-BN8992KC:~/OS$ nano W14.  
arnav@LAPTOP-BN8992KC:~/OS$ nano W14.c  
arnav@LAPTOP-BN8992KC:~/OS$ gcc W14.c -o W14  
arnav@LAPTOP-BN8992KC:~/OS$ ./W14  
Hello  
Hello
```

Program 5

```
#include<stdio.h>  
#include<unistd.h>  
int main()  
{  
    pid_t pid;  
    pid=fork();  
    if(pid==0){  
        printf("I am Child Process\n");  
    }  
    else{  
        printf("I am Parent Process\n");  
    }  
}
```

Output

```
arnav@LAPTOP-BN8992KC:~/OS$ nano parent.c  
arnav@LAPTOP-BN8992KC:~/OS$ gcc parent.c -o parent  
arnav@LAPTOP-BN8992KC:~/OS$ ./parent  
I am Parent Process  
I am Child Process
```

Program 6

```
#include<stdio.h>
#include<unistd.h>
int main(){
fork();
printf("Hello\n");
fork();
printf("Hello\n");
return 0;
}
```

Output

```
arnav@LAPTOP-BN8992KC:~/OS$ nano fork.c
arnav@LAPTOP-BN8992KC:~/OS$ gcc fork.c -o fork
arnav@LAPTOP-BN8992KC:~/OS$ ./fork
Hello
Hello
Hello
Hello
Hello
Hello
```

Program 7

```
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>
int main()
{
    pid_t pid;
    pid=fork();
    if(pid==0)
    {
        printf("Child Running\n");
    }
    else{
        wait(NULL);
        printf("Parent Running After Children\n");
    }
    return 0;
}
```

Output

```
arnav@LAPTOP-BN8992KC:~/OS$ nano read.c
arnav@LAPTOP-BN8992KC:~/OS$ nano wait.c
arnav@LAPTOP-BN8992KC:~/OS$ ./wait
Child Running
Parent Running After Children
arnav@LAPTOP-BN8992KC:~/OS$ |
```

Program 8

```
#include <stdio.h>
#include <unistd.h>
int main(){
    pid_t pid;
    pid=fork();
    if(pid == 0){
        printf("Child pid=%d\n",getpid());
        printf("Parent pid=%d\n",getpid());
    }
    else{
        printf("Parent pid=%d\n",getpid());
    }
    return 0;
}
```

Output

```
arnav@LAPTOP-BN8992KC:~/OS$ nano pid.c
arnav@LAPTOP-BN8992KC:~/OS$ gcc pid.c -o pid
arnav@LAPTOP-BN8992KC:~/OS$ ./pid
Parent pid=1253
Child pid=1254
Parent pid=1254
```